Privacy Protection for Marginal-Sensitive Community Individuals Against Adversarial Community Detection Attacks

Cong Chen, Zhongyuan Jiang, and Jianfeng Ma, *Member, IEEE*

***Abstract*— Social networks especially the social communities facilitate the rapid and rich social activities of all individuals in the world. However, advanced community detection brings serious privacy disclosure (e.g., whether two important and sensitive individuals belong to the same community?) to us. For instance, online plainclothes policemen can be regarded as marginal community users who are often in the same community initially and in need of penetrating into as many as possible different communities to collect illegal evidence of network criminals, but adversarial community inference which can maliciously disclose the sensitive user relationships within a target community will expose their privacy and lead to task failure. Thus, privacy protection for the marginal community users becomes an urgent issue which is still open so far. In this work, we aim to study the community privacy protection for target marginal individuals of a community against multiple adversarial community detection (ACD) attacks. First, we define the marginal community user hiding problem and propose a marginal user pair selection strategy. Second, to enhance the privacy effectiveness of conventional methods, we propose a deep graph learning approach to maximally find the minimum link perturbation cost. Finally, we conduct various community detection attacks on many real social graphs, and the experimental results show that our method can more effectively hide the marginal-sensitive user pairs than baselines.**

***Index Terms*— Adversarial detection, community detection, community perturbation, selection strategy, sensitive individuals. PACS: 89.75.Hc, 05.45.-a, 89.20.-a**

I. Introduction

COMMUNITY detection is more and more widely used in the field of complex network research, from the initial protein-related functional analyzing to the user's interest mining [1]. In the information network, similar pages of a web service can be classified together [2], [3]. In the social network, different communities can be well-discovered, and the potential relationship link between two individuals can be efficiently derived [4], [5]. Fraudulent organization detection works well in the user transaction network [6]. With increasing applications of community detection, the community-related privacy disclosure has become more and more important, which currently imperil personal privacy or security [7].

One feasible privacy protection solution is the adversarial community detection (ACD). Nagaraja [8] proposed the ACD problem which is to add edges to many highly centralized nodes in the graph to hide given sensitive individuals. Waniek et al. [9] unconnected a fraction of links within a specific community and connected several new links among communities to hide target individuals or communities. In this mechanism, they are inclined to choose the hub nodes (e.g., highly connected ones in the network). It has two main disadvantages. One is that it causes higher cost to hide hub nodes. The second is that it causes high utility loss (e.g., degree centrality) for the original network structure. Moreover, sensitive users are not necessary the hub ones, and instead, for instance, the marginal nodes that are the connection bridges (e.g., key intermediate information exchange stations to the outer social communities, which may process the most sensitive fairs) to exterior communities seem to be more sensitive. Li et al. [10] proposed a method based on graph learning to train to do link inserting or link deleting to defend community detection. The targets are many randomly selected nodes or a given specific community. This work gave out an efficient and general ACD solution, but there is also a lack of consideration of marginal user privacy protection which includes the budget-limited marginal node selection, adversarial detection modeling, and robustness evaluation against multiple general community detection attacks.

In response to the above problems, we have three challenges as follows.

1) How to efficiently select the marginal nodes?
2) How to fight against multiple community detection problems?
3) How to find the minimum budget problem with the help of graph learning methods?

For the selection of target nodes that need privacy protection, we mainly focus on the marginal nodes of a community, because in a social network, the privacy concerns of many specific individuals are nonidentical in different specific occasions.For example, many sensitive users such as

Manuscript received 31 May 2022; revised 1 October 2022 and 22 November 2022; accepted 6 December 2022. Date of publication 22 December 2022; date of current version 31 January 2024. This work was supported in part by the Key Research and Development Program of Shaanxi under Grant 2023-YBGY-270, the National Key Research and Development Program of China under Grant 2022YFB2701800, in part by the National Natural Science Foundation of China under Grant 62076191 and Grant 61502375, and in part by the Fundamental Research Funds for the Central Universities of China under Grant JB211501. *(Corresponding author: Zhongyuan Jiang.)*

The authors are with the School of Cyber Engineering and the State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an, Shannxi 710071, China (e-mail: cchen_8@stu.xidian.edu.cn; zyjiang@xidian.edu.cn; jfma@mail.xidian.edu.cn).

Digital Object Identifier 10.1109/TCSS.2022.3229162

2329-924X © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on October 27,2026 at 10:37:20 UTC from IEEE Xplore. Restrictions apply.

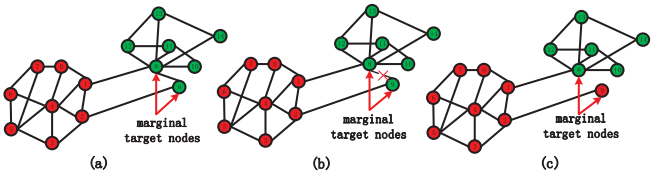


Fig. 1. Simple example of link perturbation to hide marginal target node pair community information. (a) At first, the two marginal target nodes are clustered into one community. (b) With a small perturbation of deleting an edge, the two targets are assigned into different communities by the same detection method (c).

plainclothes police officers need to penetrate into different communities [11] as ordinary users to collect the evidence of network criminal events such as network violence, service attack, and unhealthy information propagation. Moreover, in the original network, these plainclothes police officers often participate in more complex social relationships and are the marginal nodes of multiple communities (i.e., marginal community nodes), and they are also mostly in a community. Friend relationships tend to expose their teammates more easily. Attackers can use advanced community detection methods to analyze whether two individuals belong to the same community (e.g., police officer group), thereby inferring the social role of a given member. Thus, the personal privacy of these plainclothes police officers in law enforcement is likely to be detected. It may endanger the life of the plainclothes policeman. If we perturb the social network structure before the community of plainclothes police being discovered, the community detection attack will not be able to analyze whether one individual belongs to a specific career community. In other words, the individual privacy on a community can be protected.

In Fig. 1(a), for instance, using the heuristic strategy, we can choose 8 and 9 as the marginal target nodes to be attacked. As mentioned above, certain social groups in the network belong to sensitive groups, such as the network police community to which nodes 8 and 9 may belong. Here, we assume the two communities (i.e., the left one and the right one) are criminal groups, and initially the two police officers (i.e., node 8 and 9) have penetrated into the right group. To help officers successfully penetrate into every gang, in Fig. 1(b), we assume the link between 8 and 9 is disconnected, and in Fig. 1(c), the two nodes are divided into two different communities in the perturbed graph using the same community detection method. Thus, at the worse, if one of them is exposed, the other one will not be implicated by the adversarial career community inference.

Furthermore, to test the robustness of privacy protections under more widely used community detection attacks, we use multiple community detection methods to divide the network, and then heuristically collect marginal nodes corresponding to each community detection method, and use intersection ones as the target marginal nodes. Finally, we run extensive experiments on six widely used real social graphs to demonstrate the effectiveness of our methods in this work.

The rest of this article is organized as follows. Section II reviews the related worked on the community detection algorithm and ACD, and Section III describes the problem definition, model, and method. We report the experimental results in Section IV and discuss future research directions in Section V.

II. Related Work

A. Community Detection

The community detection algorithm has developed rapidly which is mainly divided into the following two categories: nonoverlapping community detection and overlapping community detection. In this work, we mainly focus on nonoverlapping community detection adversarial research.

First, the nonoverlapping community structure means that each node in the network can only belong to one community, and there is no intersection between communities. The GN algorithm [12] was proposed by Girvan and Newman [35]. It adopts the idea of edge betweenness to divide and process small networks. Diffusion is a process of propagation, which spreads information, similarity, interests to form communities. In the spectral clustering method [4], a similarity graph is constructed from the network, and then the community is determined based on the spectrum analysis of the similarity graph. Label propagation [13] is one of the typical algorithms and many algorithms are proposed based on its idea, such as Copra [14], speaker-listener label propagation algorithm (SLPA) [15]. Closeness is used as a distance metric in community detections. Walktrap and Informap are its typical algorithms [16]. A structure can be considered as graph mining, whose aim is to find the maximal structures to satisfy constraints and structural rules. The finding local community structure in networks (CNM) algorithm [17] achieves a hierarchy of the community which contains the given node by searching the graph. Later, people developed other community detection algorithms based on the above classic community detection algorithms. Imtiaz et al. [18] proposed a multilayer ant colony optimization algorithm to solve the problem of single-objective optimization that is easy to fall into local optimal solutions. For the first time, they proposed a method of optimizing multiobjective functions and applied it to community detection. Inuwa-Dutse et al. [19] proposed a new detection method based on an extensible framework, which uses community structure and text information to identify related communities in the network.

In the real world, many community structures may appear in an overlapping manner, that is, there are intersections between communities, and a node may belong to multiple communities. Palla et al. [20] proposed the clique penetration method (CPM) to find the largest cliques in the network, and then used these largest cliques to find the connected subgraph of k cliques. The community structure can be adjusted by changing the k value. Ronhovde and Nussinov [21] used the Potts model community detection algorithm to accurately and quantitatively assess the hierarchical or multiresolution structure of graphs. Zhang et al. [22] proposed a multilabel-based overlapping detection method. Whang et al. [23] proposed a seed expansion-based overlapping search method, the key idea of which is to find good seeds and then greedily expand these seeds according to community metrics. Asmi et al. [24] proposed a method for greedy coupled seed expansion-based

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on October 27,2026 at 10:37:20 UTC from IEEE Xplore. Restrictions apply.

detection, specifically by selecting a node and its most similar neighbors to construct a coupled seed, and then using the improved local community. The identified fitness function expands this coupling seed and finally achieves the purpose of detecting overlapping communities.

B. Adversarial Community Detection

With the rapid development of community detection methods, the scope of application has become more and more extensive, and personal privacy issues have gradually been exposed. How to protect the personal privacy against community detection methods has become very important. In complex networks, the degrees of the two nodes connected to a link can indicate the link strength. Attackers can attack social networks based on this feature and get the relationship between a pair of target nodes. Liu and Terzi [25] studied the modification of social networks through a set of operations of adding or deleting edges, so that they can construct this social network into a K-anonymous social network. Yuan et al. [26] proposed an anonymous method of k-degree-l-diversity, which is used to protect some sensitive attributes of individuals. In the research of Zhou and Pei [27], they add edges through node labels, to achieve $k-1$ different neighborhoods to hide target nodes.

Zheng et al. [28] studied the user information protection problem and proposed a new framework to discover the community structure in online social networks. At the same time, it can also retain sensitive link information. In this process, a third-party server dynamically requests the weight of social links to optimize the current community structure instead of directly asking about the existence of links, so as to counter many community detection methods. Fionda and Pirro [29] proposed the concept of community deception to counter community detection, and they proposed definitions of deception scores to evaluate the effects of their own methods. Chen et al. [30] proposed a strategy for rewiring a small amount of social links to achieve privacy protection of community users.

Although the research of related work can hide individuals or communities, and achieve the purpose of resisting community detection attacks by perturbing a tiny fraction of social links operations are costly. For example, Zheng et al. [28] proposed the use of genetic algorithms to achieve the detection purpose of antagonistic communities. The results show that target individuals can be well hidden, but for large networks, the efficiency and effectiveness are poor. To maximally maintain the network topology and properties, the link modifications should be as small as possible to escape the advanced community detection. Deep learning has been proven to help improve performance [10], and its efficiency and effectiveness have been significantly improved through extensive experiments. Therefore, in this work, we introduce deep learning into the community user pair hiding of marginal users. In the experiments, we perturb the generated confrontation graph by the graph learning based on the actor-critic framework and set a threshold for the perturbation size. After continuous learning and training, the cost of perturbing the graph is relatively small.

Our method differs from the methods of Nagaraja [8] and others [9], [29] mainly in the following aspects. First, the hidden object we choose, or the attack object is more purposeful (i.e., marginal community nodes). Their methods are mainly to randomly select nodes as the attack objects. Second, we use a deep graph community perturbation mechanism to hide the community relationships of target user pairs. With this method, we spend much smaller budgets than their method [8], [9], [29]. Third, the hidden objects we selected are calculated by multiple community detection methods, so that they can defend against multiple community detection methods.

III. Problem Definition, Models and Methods

A. Problem Definition

To solve the problem of how to select sensitive community marginal nodes and generate a confrontation graph at a relatively low cost, we designed the following model. We define a network graph $G = (V, A, Y)$, where $V = (v_1, v_2, \ldots, v_n)$ represents the node set in the network. A is the adjacent matrix of the network where $A_{ij} = 0$ or 1 indicates whether there is an edge connection between nodes i and j. We use $Y = \{y_1, y_2, \ldots, y_n\}$ to represent the attribute value of the node, and $y_i \in R^d$ is a d-dimensional vector.

The community detection problem aims to partition a graph G into M disjoint subgraph $G_i = (V_i, A_i, Y_i), i = \{1, \ldots, M\}$, where $V = \cup_{i=1}^{M} V_i$ and $V_i \cap V_j = \phi$ for $i \neq j$. In the context of community detection, we are interested in a community marginal node set $S^+ \subset V$ that actively hopes to evade detection. The marginal community node set, for example, undercover agents want to penetrate into the enemy's interior and therefore want to hide another identity of themselves, or plainclothes police officers want to hide the identity of the police to engage in investigation activities. Our goal is to hide these marginal node pairs with smaller disturbances. A smaller disturbance is to ensure that the disturbance is imperceptible, so we set a threshold to reflect the magnitude of the disturbance. Given a community detection algorithm D, define the community detection against attack problem as learning the attacker function g to make small disturbances to G resulting in $\hat{G}$. Fig. 2 shows the realization principle of our scheme; for this, we state our problem as follows:

$$\begin{aligned} &\max\ (\zeta(D(G), S^+) - \zeta(D(\hat{G}), S^+)) \\ &\text{s.t. } \hat{G} \longleftarrow \arg\min g(D, (G, S^+)) \\ &\quad Q(G, \hat{G}) < \Delta \end{aligned} \tag{1}$$

where $\zeta(\cdot, \cdot)$ is used to reflect a measure of the community detection results after the community is disturbed, and $Q(G, \hat{G}) < \Delta$ is used to ensure that the disturbance is within a given threshold range so that we can find a suitable disturbance. In this experiment, we mainly perturb the insertion or deletion of the edges of the sensitive community, i.e., $\hat{G} = (\hat{V}, \hat{A}, \hat{Y}) = (V, \hat{A}, Y)$ at the lowest rewiring cost. Table I describes all the symbols used in this article.

B. Model Framework

In our problem setting, we have three main modules: 1) a marginal community user selection algorithm to select the target-sensitive nodes; 2) an adversarial attacker $g(\cdot)$ that attacks the links adjacent to selected nodes; and 3) the

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on October 27,2026 at 10:37:20 UTC from IEEE Xplore. Restrictions apply.

TABLE I
ADOPTED SYMBOLS

symbols	Definition
G	A social graph
V	The node set of a network
E	The edge set of a network
$g(\cdot)$	An attacker function
A	The adjacency matrix
Y	The attribute values
M	The number of communities
N	The number of nodes
$m_\phi(X\|A,Y)$	The used for encoding
ϕ	The empty set
S^+	The community marginal node set
$D(G)$	A community detection algorithm
$\hat{G}$	A(An updated) social network
ζ_{feed}	The combined loss
$\zeta(.,.)$	Measure of community disturbance
$Q(G,\hat{G})$	The limit perturbation value function
ζ_{deviat}	The unsupervised loss value
S	The community allocation matrix
$Tr(\cdot)$	The sum of the element on the main diagonal
$\|\ .\ \|_F$	Represents the Frobinius norm of the matrix
I_M	An identity matrix
$\overline{A}$	Normalize the adjacency matrix
$\sigma(\cdot)$	An activation function
W_1, W_2, W_{s1}, W_{s2}	Weight matrices
$\zeta_{p-constraint}$	The perturbation loss
$K(\cdot\|\|\cdot)$	The Kullback-Leibler divergence
$UN(\cdot)$	Representation of neighboring nodes
ϕ	The encoding parameter set
θ	The generation parameter set
$\zeta_{distribut}$	The smallest distance for any pair inside S^+
$\zeta_{p-constraint}$	The value of perturbation limit condition
C	Deleted edge
$\bar{C}$	Inserted edge

community detection algorithm $D(\cdot)$ to detect communities of the attacked graph and the original graph and calculate the similarity between them. Modules 2) and 3) are highly correlated, i.e., the adversarial attack function needs the feedback of the community detection algorithm $D(\cdot)$ to test whether the marginal node set achieves the purpose of hiding. The community detection algorithm $D(\cdot)$ relies on adversarial samples to enhance its robustness. However, it is difficult to use the gradient of the loss function to achieve our goal in judging the loss of the graph in discrete data of the graph. Inspired by the research of [31], we use the gradient strategy in the actor-critic framework as two modules of the interaction between signals to solve the problem. Another point is that we need to design a surrogate community detection model to detect the generated adversarial graphs. In our solution, we design an iterative framework consisting of two neural networks, one serving as an adversarial graph generator $g(\cdot)$ and the other as a surrogate community work detection method $D(\cdot)$. Fig. 2 depicts the framework. The upper part generates the adversarial graph, which is optimized with respect to the optimum of the surrogate detection model in the lower part. However, when instantiating the two neural networks, we face three challenges as follows.

1) *Community Detection Model:* Whether a generated graph which initially defends one community detection can defend other community detection attacks?

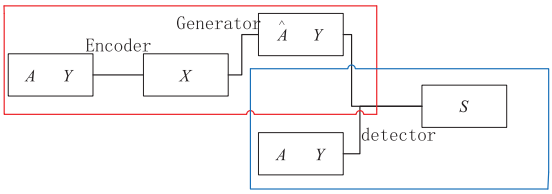


Fig. 2. Overview of our model framework. The adversarial graph generator (in red) produces a constrained adversarial graph, which is used to train a robust surrogate community detection model (in blue).

2) *Minor Disturbance:* What criteria should we use to ensure that the modification is so small that it cannot be noticed by the user?
3) *Constrained Graph Generation:* How to generate adversarial graphs that effectively satisfy tiny perturbations?

The issues involved in the above three modules make our problem very complicated. Next, we present solutions to the challenges involved in the three modules.

Algorithm 1 Community Marginal Nodes

Input: G;
Output: $Marginal_set$;
1: $comm1 \longleftarrow G.community_informap()$; // Community divisions by Informap
2: $comm2 \longleftarrow community.best_partition()$; // Community divisions by Louvain
3: $comm3 \longleftarrow G.community_edge_betweenness()$; // Community divisions by GN
4: set_comm1, $set_comm2 \longleftarrow set_comm(comm1)$, $set_comm(comm2)$; // Get marginal nodes from comm1 and comm2
5: $set_comm3 \longleftarrow set_comm(comm3)$; // Get marginal nodes from comm3
6: $set_comm1to2 \longleftarrow Getinter(set_comm, set_comm2)$; // Get intersection between set_comm1 and set_comm2
7: $S^+ \longleftarrow Getinter(set_comm1to2, set_comm3)$; // Get intersection set S^+ between $set_comm1to2$ and set_comm3

C. Marginal Nodes' Selection Strategy

Algorithm 1 implements the marginal node selection strategy. First, we use different community detection methods to divide the social network into communities (lines 1–3). Then we calculate the shortest distance between each node and the adjacent community using the Dijkstra [32] algorithm, and nodes with the shortest distance within 2 are collected into the marginal community node set for each used community detection method (lines 4 and 5). We find out the intersection nodes of multiple above sets as the target marginal user set S^+ (lines 6 and 7). We then conduct our experiments on the selected S^+. For instance, under the Informap community detection algorithm, it specifies different communities for each vertex and iteratively merges nodes according to modular gain. If there is no gain, the node will remain in its own

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on October 27,2026 at 10:37:20 UTC from IEEE Xplore. Restrictions apply.

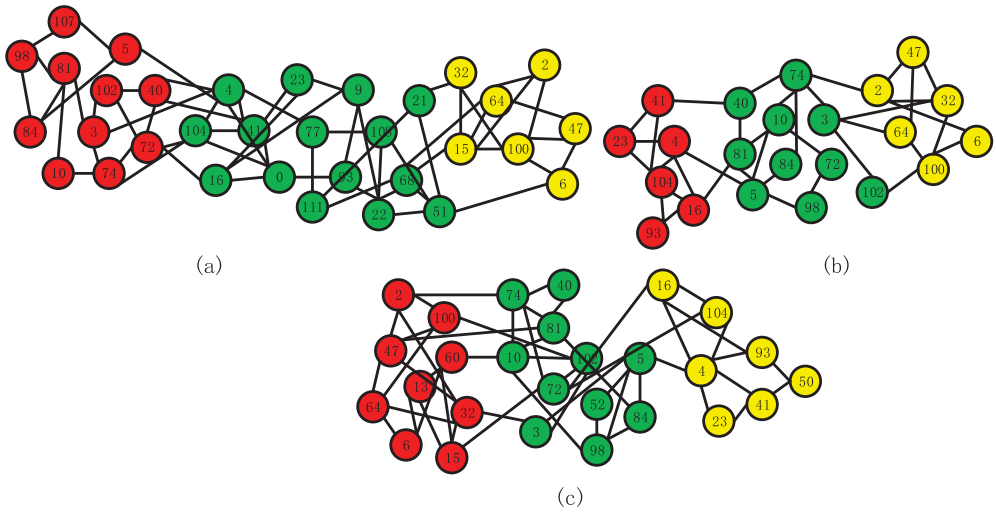


Fig. 3. Community divisions under (a) Louvain, (b) GN, and (c) Informap community detection algorithms on a part of the Football dataset.

community; repeat this process until the modular gain does not increase. Then, it reconstructs the network by replacing the community determined in the first stage with a super node to continue the iterative calculation in the second stage and so on. Its time complexity is $O(n\log n)$. We use the Dijkstra [32] algorithm to find the shortest path between nodes. The time complexity of the Dijkstra [32] algorithm is $O(n^2)$, so the total time complexity of our algorithm is $O(n^2)$. The specific implementation is as follows.

In module 1, our purpose is to select marginal community nodes as hidden objects, but the number of communities obtained by different community detection methods and the nodes in the community will be different, so we obtain common marginal nodes shared by multiple communities through marginal intersection set S^+ and hide these nodes in set S^+ so that the generated adversarial graph can fight against these several community detection methods. Fig. 3 shows part of the community and part of the nodes obtained by dividing the Football graph [33] using three community detection algorithms, i.e., Louvain [34], GN [35], and Informap [36]. In Fig. 3(a), we use the partial node division graph obtained by the Louvain community detection method, and the set of marginal nodes that we choose is $L_1 = \{5, 40, 72, 74, 3\}$, $L_2 = \{4, 104, 16, 41, 21, 111, 51, 68\}$, and $L_3 = \{32, 15, 64, 6\}$ corresponding to the above three divided communities, respectively. In Fig. 3(b), we use the partial node division graph obtained by the GN community detection method. The set of community marginal nodes we choose is $G_1 = \{41, 4, 16\}$, $G_2 = \{40, 81, 5, 74, 3, 102\}$, and $G_3 = \{2, 32, 64, 100\}$. Similarly, in Fig. 3(c), the graph is divided by the informap community detection method, the marginal node set selected by the community is $I_1 = \{2, 100, 47, 60, 32, 15\}$, $I_2 = \{74, 81, 10, 72, 3, 5, 84\}$, and $I_3 = \{16, 104, 4\}$. L_1, G_2, and I_2 have an intersection $s_1 = \{55, 74, 3\}$, while L_2, G_1, and I_3 have intersections $s_2 = \{4, 16\}$. Moreover, L_3, G_3, and I_1 have an intersection $s_3 = \{32\}$. For the selection of the remaining marginal nodes, we try to choose those nodes distributed in multiple communities. For example, L_3, G_3, and I_1 have an intersection LG $= \{32, 64\}$, LI $= \{32, 15\}$, and GI $= \{2, 32, 100\}$, except for the intersection of those marginal nodes. Finally, we get the community marginal node set S^+ that we need.

D. Community Detection Model

We propose a graph-learning-based community detection model which has two key issues: 1) a distance function that measures the quality of results, corresponding to the loss function used in neural networks, and 2) a method for detecting communities, which corresponds to the neural network architecture. We discuss these two issues accordingly below.

1) Loss Function: To know the size of the disturbance during the process of community disturbance, we use the loss function defined by [37] to calculate the size of the loss

$$\frac{1}{M}\sum_{m}\frac{\text{cut}(V_m, \overline{V_m})}{\text{vol}(V_m)} \tag{2}$$

where $\text{vol}(V_m) = \sum_{i\in V_m}\text{degree}(i)$, $\text{degree}(i)$ is the degree of the node i, $\overline{V_m} = V\backslash V_m$, and $\text{cut}(V_m, \overline{V_m}) = \sum_{i\in V_m, j\in\overline{V_m}} A_{ij}$.

We follow the derivation method in [18] to obtain the following unsupervised loss function:

$$\zeta_{\text{deviat}} = -\frac{1}{M}\text{Tr}((S^T AS)\Theta(S^T BS)) + \gamma\left\|\frac{M}{N}S^T S - I_M\right\|_F^2 \tag{3}$$

where $S \in R^{N\times M}$ is the community allocation matrix, B is the degree matrix $B_{ii} = \sum_j A_{ij}$, Θ is the elementwise division, and $\text{Tr}(\cdot)$ is defined as the sum of the elements on the main diagonal of a given square matrix. We set $\gamma = 0.1$, $\|\cdot\|_F$ represents the Frobinius norm of the matrix, where I_M is an identity matrix, M represents the number of community divisions, and the application of (M/N) is due to standardized cutting to stimulate balanced clustering and gap reduction deviation [37], [38].

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on October 27,2026 at 10:37:20 UTC from IEEE Xplore. Restrictions apply.

2) Network Architecture: This part introduces the use of the neural network architecture to derive the community allocation matrix S, which is trained through the unsupervised loss function obtained in (3). For the community detection architecture, we use a popular graph neural network (GNNS) [39], [40] architecture, as shown in Fig. 2. This architecture has two parts: 1) embedding nodes and 2) assigning communities. In the first part, we use GNNS to represent the nodes, and then in the second part, we assign similar nodes to the corresponding communities, thus classifying the communities.

For the part of embedding nodes, we follow the graph convolutional network GCNs [39] to achieve.

First, we normalize the adjacency matrix

$$\overline{A} = \frac{1}{\sqrt{\bar{B}}}(I_N + A)\frac{1}{\sqrt{\bar{B}}} \tag{4}$$

where I_N is the identity matrix, and $\bar{B_{ii}} = \sum_j (A + I_N)_{ij}$, and then use the two-hop smoothing method on the graph structure to transform the characteristics of the node to get

$$L_l = \overline{A}\sigma(\overline{A}YW_1)W_2 \tag{5}$$

where $\sigma(\cdot)$ is an activation function, and $W_1 \in R^{d\times h}$ and $W_2 \in R^{h\times v}$ are the weight matrices. After getting the result L_l of the node representation, we can use the softmax normalized exponential function to divide the result, that is, divide similar nodes into the same community, and the formula is as follows:

$$S = \text{softmax}(\sigma(L_l W_{s1})W_{s2}) \tag{6}$$

where $W_{s1} \in R^{v\times r}$ and $W_{s2} \in R^{r\times M}$ are the weight matrices. The function of W_{s1} is to linearly transform the node representation from v-dimensional space to r-dimensional space, and then introduce nonlinearity through function σ. W_{s2} is used to assign scores to each of the M communities. Then softmax is applied to derive the standardized distribution of each node in M communities, which means that the sum of the M scores of each node is 1.

To summarize, we design a neural network to serve as the surrogate community detection model, which is trained in an unsupervised way based on the loss function ζ_{deviat}.

E. Minor Disturbance

Given G and $\hat{G}$, we know the correspondence between nodes in two graphs, to be able to generate adversarial graphs with less disturbance to the original community and described in Fig. 4. The degree of perturbation is determined by the following perturbation losses:

$$\sum_{i<j} |A_{ij} - \hat{A}_{ij}| \le \Delta \tag{7}$$

$$\zeta_{\text{p-constraint}} = \sum_{v_j} K(\text{UN}(v_j|(G, A, Y)) \times \|\text{UN}(v_j|(\hat{G}, \hat{A}, \hat{Y}))) \tag{8}$$

where $K(\cdot\|\cdot)$ is the Kullback–Leibler divergence. $\text{UN}(\cdot)$ is to learn the representation of neighboring nodes in an unsupervised way.

We also use the methods from variational automatic coding [10] and use some parametric latent variable models to generate $\hat{A}$ we need.

F. Constrained Graph Generation

In this section, we describe the generation of A that satisfies the budget constraint (7) and the perturbation loss constraint (8) as follows:

$$P(\hat{A}|A, Y) = \int m_\phi(X|A, Y)n_\theta(\hat{A}|A, Y, X)dX \tag{9}$$

where $m_\phi(X|A, Y)$ is used for encoding, ϕ is the encoding parameter set, $n_\theta(\hat{A}|A, Y, X)$ is the generation part, and θ is the generation parameter set. But for some of the budget constraint problems involved in the above formula, we have solved it from [25], and we get the approximate expression of the above formula

$$m_\phi(X|A, Y) = \prod_{i=1}^{N} m_{\phi_i}(x_i|A, Y) \tag{10}$$

$$n_\theta(\hat{A}|A, Y, X) = \prod_{(i,j)\in C} \Theta(E_{ij}) \prod_{(i,j)\in \bar{C}} \Psi(E_{ij}) \tag{11}$$

$$\zeta_{\text{feed}} = \zeta_{\text{precur}} + (\alpha\zeta_{\text{distribut}} + \beta\zeta_{\text{p-constraint}}) \times \left(\sum_{(i,j)\in C} \log\Theta(E_{ij}) + \sum_{(i,j)\in \bar{C}} \log\Psi(E_{ij})\right) \tag{12}$$

where $m_{\phi_i}(x_i|A, Y)$ is the predefined prior distribution, namely, isotropic Gaussian with diagonal covariance. $\zeta_{\text{precur}} = K(m(X|Y, A)||n(X))$ is the prior loss with $n(X) = \prod_{i=1}^{N} N(x_i|0, I)$, and $\zeta_{\text{p-constraint}}$ is a limiting condition of perturbation in (8). C represents the set of $\Delta/2$ edges to be deleted, and $\bar{C}$ indicates that the $\Delta/2$ edges set to be inserted has been selected. In the experiment, we have to select Δ edges in the initial graph for processing. For our datasets DBLP and Finance, for small-scale graphs, we divide the budget Δ into two parts: $\Delta/2$ for deleting edges and $\Delta/2$ for inserting edges. For medium-sized graphs, we dedicate budget Δ entirely to deleting edges

$$\zeta_{\text{distribut}} = \min_{i\in S^+, j\in S^+} K(S_{i,:}\|S_{j,:}) \tag{13}$$

where $\zeta_{\text{distribut}}$ regarded as the smallest distance for any pair inside S^+, $\alpha < 0$, and β aims to maximize the margin so that the members of S^+ are spread out across the communities in $\cup_{i=1}^{M} G_i$.

$E_{ij} = [X_i|Y_i]\odot[X_j|Y_j]$ in (11), $\odot$ represents multiplication calculated by element, and $X_i|Y_i$ is the correlation of X_i and Y_i. $\Theta(\cdot)$ is defined as follows:

$$\Theta(E_{ij}) = \frac{e^{b_{ij}}}{\sum e^{b_{ij}}}, \quad \text{if } A_{ij} = 1 \tag{14}$$

$$b_{ij} = W_{b1}\sigma(W_{b2}E_{ij}), \quad \text{if } A_{ij} = 1 \tag{15}$$

where $\Theta(E_{ij})$ is calculated by the softmax function and the double-layer perceptron. $\Psi(E_{ij})$ is computed in the same way as $\Theta(E_{ij})$ with nonidentical parameters, and the essential differences are: 1) we compute with the condition of $A_{ij} = 0$ and 2) we add $\Delta/2$ edges to $\bar{C}$.

To summarize, (12) receives error signals $\zeta_{\text{distribut}}$ and $\zeta_{\text{p-constraint}}$ from the surrogate community detection model, which serves as a reward to guide the optimization of our graph generator.

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on October 27,2026 at 10:37:20 UTC from IEEE Xplore. Restrictions apply.

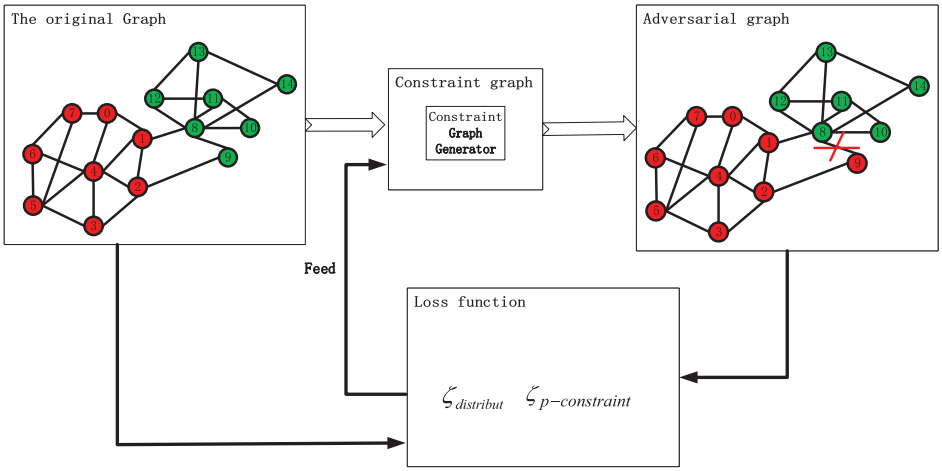


Fig. 4. Constrained graph generation and surrogate community detection which detects the generated confrontation graph and original graph. Under the optimization of the latter, it will produce two kinds of losses to guide further generation of constraint graph.

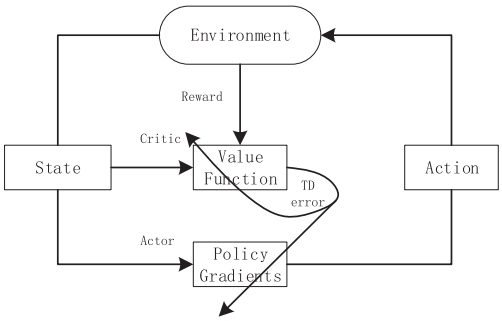


Fig. 5. Actor-critic framework diagram.

G. Basic Framework

1) *Actor-Critic Framework [31]:* It is composed of two modules, one is an actor and the other is an evaluator, as shown in Fig. 5. The actor uses the gradient function to generate actions and interact with the environment. For critic, the value function is used to evaluate the performance of the actor. Simply put, actor-critic is the time difference (TD) version of the policy gradient [41]. The actor decides which action should be taken during the action, and the critic tells the actor how good his performance is and how to adjust in the next step. Our framework adopts this type, and the framework model is as follows.

From the perspective of the actor-critic framework, the attacker adversarial function $g(\cdot)$ could be considered as the actor and the community detection algorithm $D(\cdot)$ could be regarded as the critic. $g(\cdot)$ attacks the network graph to generate an adversarial graph, and then $D(\cdot)$ detects the adversarial graph to get some error signals, and then feeds these error signals back to $g(\cdot)$, $g(\cdot)$ according to the results obtained to carry out improvement attacks and alternate between them until you get the best result. The specific training process is shown in Fig. 4 and Algorithm 2.

At the beginning of Algorithm 2, we use the constraint graph generator $g(G, \Delta)$ to obtain the adversarial graph $\hat{G}$ (line 3). Then, using the idea of robust training, feed G and $\hat{G}$ into the agent community detection model $D(\cdot)$ to calculate ζ_{deviat} (the first few lines). According to the optimal value of $D(\cdot)$, we obtain $\zeta_{\text{distribut}}$ and $\zeta_{\text{p-constraint}}$ to provide motivation for the learning process of $g(\cdot)$ (lines 6 and 7).

Algorithm 2 Model Training Algorithm

Input: G, Δ, X, A;
Output: $\hat{G}$;
Initial ΔD, ΔF, $Echo$;
2: **while** $echo \leq Echo$ **do**
$\hat{G} = g(G, \Delta)$
4: $\zeta_{precur} = K(m(X|Y, A)\|n(X))$
$\zeta_{deviat} = D(\hat{G}, G)$
6: $\zeta_{distribut} = \min_{i\in S^+, j\in S^+} K(S_{i,:}\|S_{j,:})$
$\zeta_{p-constraint} = \sum_{V_j} K(UN(V_j|(G, A, Y))\|UN(V_j|\hat{G}, \hat{A}, \hat{Y}))$
8: $\Delta F = \Delta F \cdot \zeta_{feed}$
$\Delta D = \Delta D \cdot \zeta_{deviat}$
10: **end while**

H. Complexity

The node set generation (i.e., which edges can be changed). The score function can be calculated incrementally and use the sparsity of the graph to ensure scalability. The runtime complexity of the algorithm can be determined as: $O(|V| \cdot |E| \cdot T_{v_0} \cdot \Delta)$ where T_{v_0} indicates the size of the two-hop neighborhood of the node v_0 during the run of the algorithm. In many iterations of each Δ. For edge perturbation, since there are two convolution layers, it is necessary to update the two-hop neighborhood of the target. Assuming the graph is sparse, T_{v_0} is much smaller than $|V|$.

IV. Experimentation and Experimental Evaluation

We construct six datasets from the DBLP and Finance real datasets to verify the effectiveness of our model: 1) DBLP_small; 2) DBLP_medium; 3) DBLP_large; 4) Finance_small; 5) Finance_medium; and 6) Finance_large. Table II lists the statistics of the six graphs.

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on October 27,2026 at 10:37:20 UTC from IEEE Xplore. Restrictions apply.

1) *DBLP Dataset:* From the DBLP bibliographic data, we constructed three co-authorship maps named DBLP_small, DBLP_medium, and DBLP_large of 304, 1304, and 20 814 authors, respectively. For each author in the DBLP dataset, we use a one-hot representation with 305 dimensions to encode his/her research keywords. Therefore, two authors i and j cooperated at least one paper, denoting $A_{ij} = 1$ and $A_{ji} = 1$ where A is the network matrix.
2) *Finance Dataset:* Finance dataset is a collection of anonymous user–user transaction data provided by Tencent. We selected 1507, 3408, and 20 121 users to establish three transaction networks Finance_small, Finance_medium, and Finance_large, respectively. For each user, we collect seven characteristics. If two users have one or more transaction records, set $A_{ij} = 1$ and $A_{ji} = 1$ in the network matrix.

A. Used Community Detection Methods

We defend the community detection attacks by hiding sensitive marginal community node pairs. We first use a variety of community detection algorithms such as Louvain [34], Informap [36], and GN [35] to find the marginal community node set of a social network. When selecting these node pairs, there may exist such case that the target marginal community user set is empty. In this case, we use other community detection methods to select many marginal users into its set.

1) *Louvain Algorithm [34]:* Louvain algorithm is an algorithm based on multilevel optimization modularity. Compared with other community detection algorithms, it can quickly and accurately divide communities.
   a) First, consider each node in the network graph as an independent community. At first, the number of communities is the same as the number of nodes.
   b) For each node i, try to assign node i to each of its neighbors in turn the community where the node is located, calculate the modularity change ΔQ before and after allocation, and record the neighbor node with the largest ΔQ. If $\max \Delta Q > 0$, assign node i to the community where the neighbor node with the largest ΔQ is located; otherwise, it remains unchanged.
   c) Repeat b) until the communities of all the nodes no longer change.
   d) Compress the graph and then compress all the nodes in the same community into a new node, then convert the weights of the edges between the nodes in the community into the weights of the rings of the new nodes, and convert the weights of the edges between the communities into new nodes edge weight between.
   e) Repeat a) until the modularity of the entire graph no longer changes.
2) *GN Algorithm [12]:* GN algorithm is the first algorithm of the community detection method, and Newman et al. [15] pioneered this field. Its idea is to find a way to delete the edges used to connect between communities. To find these edges, they use the shortest path method to solve this problem. In order to find these edges, they use the shortest path method to solve this problem and define the concept of betweenness to find these edges. In the network, for all the nodes, the number of shortest paths pass through this edge. The more the number of passes through this edge, the greater the betweenness of this edge. For the edge with high betweenness, the greater it is. It may be an edge between communities, because for nodes between two communities, their shortest path must pass through the edge between communities. The whole process is a bottom-up process, and this process finally gets a tree graph, that is, the leaf nodes of the tree represent the vertices in the network, and each layer of the tree corresponds to a specific division of the network. From all the hierarchical divisions of the tree diagram, the division with the largest modularity value is selected as the effective division of the network.
3) *Informap Algorithm [36]:* This algorithm is based on network graphs and coding theory. It detects communities by compressing the description of the information flow on the network. In other words, the key to Informap is to minimize the average description length $L(M)$, because more information is generated when randomly walking between communities. The specific steps of the Informap algorithm are as follows.
   a) Initialize, treat each node as an independent group.
   b) Randomly sample a sequence of nodes in the graph, try to assign each node to the community where the neighbor node is located in order, and assign the community with the largest average bit drop to the node. If there is no drop, the community of the node remains unchanged.
   c) Repeat until step 2 until $L(M)$ can no longer be optimized.

B. Evaluation Metrics

In the field of community detection, modularity [15] is a widely used metric to assess the quality of community partitioning. In our experiments, we use it to evaluate whether the community structure between the adversarial graph and the original graph has changed significantly or whether the purpose of small interference is achieved. The algorithm formula of modularity is as follows:

$$Q = \frac{1}{2|E|}\sum_{i,j}(A_{ij} - D_{ij})\sigma(c_i, c_j) \tag{16}$$

$$D_{ij} = \frac{1}{2|E|}\text{degree}(i)\text{degree}(j) \tag{17}$$

where $|E|$ is the number of edges in the network, and A represents the adjacency matrix. When node i and node j belong to the same community, $\sigma(c_i, c_j) = 1$; otherwise, $\sigma(c_i, c_j) = 0$, and degree(i) is the degree of node i. In essence, the degree of modularity represents the degree of similarity between the network structure and a group of communities, so in some random networks, the value of modularity may be relatively small. When the modularity of the original graph and

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on October 27,2026 at 10:37:20 UTC from IEEE Xplore. Restrictions apply.

TABLE II
STATISTICS OF GRAPHS

	$\|V\|$	$\|E\|$	No. of features	Mean degree	Mean clustering	Training steps	Test steps
DBLP_small	304	1980	305	13.02	0.72	Auto	1000
DBLP_medium	1304	6100	305	9.35	0.63	Auto	1000
DBLP_large	20814	119854	305	11.51	0.54	Auto	1000
Finance_small	1507	1237	7	1.64	0.11	Auto	1000
Finance_medium	3408	3869	7	2.27	0.13	Auto	1000
Finance_large	20121	23732	7	2.36	0.12	Auto	1000

TABLE III
PERFORMANCE COMPARISON OF MODULARITY UNDER DIFFERENT ATTACKS MEASURED UNDER THE GN COMMUNITY DETECTION ALGORITHM

Date sets	DBLP_small	DBLP_medium	DBLP_large	Finance_small	Finance_medium	Finance_large
Modularity	Q	Q	Q	Q	Q	Q
$\Delta = 2$	0.796786	0.944782	0.981237	0.890781	0.866572	0.908323
$\Delta = 4$	0.796715	0.944813	0.981783	0.891047	0.866328	0.908623
$\Delta = 6$	0.796892	0.944763	0.981982	0.891147	0.866504	0.908522
$\Delta = 8$	0.797462	0.944786	0.981879	0.881077	0.866611	0.908473
$\Delta = 10$	0.797484	0.944904	0.981732	0.889753	0.867232	0.908342
$\Delta = 12$	0.796715	0.945047	0.982478	0.888508	0.866063	0.908847
$\Delta = 14$	0.795695	0.944840	0.982722	0.884672	0.865860	0.909384
$\Delta = 16$	0.796729	0.944808	0.982674	0.891021	0.865593	0.910289
$\Delta = 18$	0.797308	0.945582	0.982238	0.894670	0.865413	0.910376
$\Delta = 20$	0.796124	0.944859	0.981378	0.891116	0.865161	0.910361
orginal Q	0.797179	0.944891	0.981236	0.909249	0.883331	0.910677

the modularity of the adversarial graph are not much different, it means that the community detection method cannot detect the difference between the two, and it achieves the purpose of minor interference; otherwise, the generated adversarial graph destroys the original network structure.

According to the technical theory in [9], we evaluate the degree of hiding of the marginal nodes of the community based on the following two evaluation values:

$$N'(S^+, G) = \frac{|G_i : G_i \cap S^+ \neq \Phi| - 1}{\max(M-1, 1) \times \max_{G_i}(|G_i \cap S^+|)}. \quad (18)$$

This scheme is used to evaluate the distribution of members of S^+ in the community. In other words, the larger the value of $N'(S^+, G)$, the better the hiding of the members in S^+

$$N''(S^+, G) = \sum_{G_i : G_i \cap S^+ \neq \Phi} \frac{|G_i \backslash S^i|}{\max(1, N - |S^+|)}. \quad (19)$$

This method is used to evaluate the degree of hiding of the members of S^+ in the community. If there are more members in the community, the lower the probability that the members of the number S^+ are assigned to the community to be discovered, the larger the $N''(S^+, G)$, then node S^+ is better hidden.

C. Attack Methods

1) *DICE [42]:* Which is a heuristic attack strategy. It first removes some edges related to the target set S^+, and then spends the remaining budget on inserting edges between S^+ and the rest of the graph.
2) *Random Target Attack (RTA):* Which follows the idea of RND [43]. Given S^+, at each step we randomly sample a node. If the node is connected to S^+, we randomly remove an edge between the node and S^+; otherwise, we randomly add an edge between the node and S^+.

D. Setup

For all the datasets, we first use Informap [36] to divide the communities, and the number of communities M for DBLP_small, DBLP_medium, DBLP_large, Finance_small, Finance_medium, and Finance_large datasets is 5, 8, 8, 5, 8, and 10, respectively. For DBLP_small, we use the marginal node selection algorithm to select five marginal nodes as target nodes in each community, and then select five nodes with the largest degree, five nodes with the largest betweenness, and five random ones as comparison objects. For other datasets, we select the target node in the same way. The budget Δ is set to 10.

In this experiment, we conducted experiments based on the installation of tensorflow1.13.1 in the pycharm platform. For adversarial graphs' generation in our experiment, we use optimizer Adam to train our model. We set $\gamma = 0.1$, the output dimension of the first layer of GCN is set to 32, and the output dimension of the second layer is set to 16. At the same time, we set the initial learning rate to 0.001, so that we can fully train our model, set 32 linear units in the fully connected layer, and set the dropout rate to 0.5.

E. Attack Performance

Table III lists the results of modularization experiments on four datasets under different budgets. In experiments, with budget Δ set from 2 to 20, we check the small perturbation of our method by: 1) running our method and obtaining the corresponding adversarial graph and 2) computing the modularity of the corresponding adversarial graph and compare it with the original one. The result of modularity is as follows.

In Table III, it can be seen that for the DBLP dataset, the modularity measured by the adversarial graphs obtained under different attacks budgets is not much different from the modularity measured of the original graph. For the Finance dataset, the adversarial graphs obtained under different attacks have a bit changes in the measured modularity and the original graph. For the Finance_small dataset, the measured modularity of the adversarial graph obtained under the attack budget $\Delta = 8$ is 0.88107, which is two percentages away from the original graph modularity of 0.90924. In the test results of the Finance_medium dataset, the measured modularity of the adversarial graph obtained under the attack budget

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on October 27,2026 at 10:37:20 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
PERFORMANCE COMPARISON OF MODULARITY UNDER DIFFERENT DETECTION METHODS

Data sets	DBLP_small		DBLP_medium		DBLP_large		Finance_small		Finance_medium		Finance_large	
	Q_1	Q_2	Q_1	Q_2	Q_1	Q_2	Q_1	Q_2	Q_1	Q_2	Q_1	Q_2
$infomap$	0.75275	0.75523	0.87885	0.87878	0.94682	0.94541	0.84630	0.83697	0.86386	0.83037	0.94503	0.94371
GN	0.79717	0.79748	0.94489	0.94490	0.98173	0.98283	0.90924	0.88975	0.88333	0.86723	0.90834	0.88763
$louvain$	0.79717	0.79748	0.94615	0.94574	0.98519	0.98483	0.89098	0.88975	0.89630	0.86723	0.90372	0.88352

TABLE V
PERFORMANCE COMPARISON AGAINST DIFFERENT TARGET ATTACKS ON THE DBLP_SMALL DATASET

Data sets	Marginal		Random		Maximum		Betweenness	
	$N^{'}$	$N^{''}$	$N^{'}$	$N^{''}$	$N^{'}$	$N^{''}$	$N^{'}$	$N^{''}$
$\Delta = 2$	0.070833	**0.880000**	0.075000	0.810000	0.064286	0.740000	0.076190	0.610000
$\Delta = 4$	**0.125833**	**0.840000**	0.580000	0.780000	0.085714	0.800000	0.047619	0.530000
$\Delta = 6$	**0.183333**	**0.720000**	0.095833	0.680000	0.045238	0.680000	0.026190	0.600000
$\Delta = 8$	**0.145833**	0.700000	0.137500	0.470000	0.095238	0.700000	0.073810	0.390000
$\Delta = 10$	**0.108333**	**0.690000**	0.091667	0.660000	0.035714	0.410000	0.078571	0.460000
$\Delta = 12$	0.112500	**0.840000**	0.112500	0.510000	0.080952	0.770000	0.038095	0.550000
$\Delta = 14$	**0.129116**	0.720000	0.108333	0.670000	0.042857	0.730000	0.054762	0.500000
$\Delta = 16$	**0.104167**	0.780000	0.750000	0.870000	0.080952	0.720000	0.076190	0.560000
$\Delta = 18$	**0.187500**	0.660000	0.058000	0.590000	0.069048	0.780000	0.057143	0.510000
$\Delta = 20$	**0.108333**	**0.920000**	0.075000	0.710000	0.097619	0.600000	0.047619	0.480000

TABLE VI
PERFORMANCE COMPARISON OF DIFFERENT NODE SETS UNDER DIFFERENT DATASETS

Data sets	DBLP_small		DBLP_medium		DBLP_large		Finance_small		Finance_medium		Finance_large	
	$N^{'}$	$N^{''}$	$N^{'}$	$N^{''}$	$N^{'}$	$N^{''}$	$N^{'}$	$N^{''}$	$N^{'}$	$N^{''}$	$N^{'}$	$N^{''}$
Maximum	0.03571	0.41000	0.64158	0.73437	0.05332	**0.78618**	0.06190	0.57000	**0.03659**	0.51694	0.03309	0.78320
Random	0.09166	0.66000	0.04693	0.72410	0.03283	0.76323	**0.10416**	0.61999	0.02742	0.58433	0.03240	0.81259
Betweenness	0.07857	0.46000	0.05170	0.83258	0.04727	0.78389	0.05416	0.48000	0.03290	0.54624	0.03629	0.67012
Marginal	**0.10833**	**0.69000**	**0.11770**	**0.84151**	**0.08109**	0.57619	0.07291	**0.73499**	0.03319	**0.62734**	**0.04139**	**0.87393**

TABLE VII
COMPARISON OF EXECUTION TIME OF DIFFERENT ATTACK METHODS UNDER DIFFERENT DATASETS (IN MINUTES)

Attack methods	ACD	RTA	DICE
DBLP_small	**1.3**	1.5	1.4
DBLP_medium	**9.4**	10.2	9.7
DBLP_large	**420.3**	450.6	432.4
Finance_small	**2.1**	2.3	2.2
Finance_medium	**8.2**	8.5	8.4
Finance_large	**430.6**	459.2	443.8

$\Delta = 20$ is 0.86516 and the difference between the original graph modularity of 0.88333 is also about two percentages. For the Finance dataset, the reason for this small change may be that the Finance dataset is a sparse graph, which has a small impact on the original graph under slight interference which is acceptable in reality. To a certain extent, the adversarial graph generated by our ACD attack does not have a large impact on the original graph and achieves the purpose of minor interference.

In Table IV, Q_1 represents the modularity obtained by different detection methods under the original graph, and Q_2 represents the modularity of the attack graph obtained under the budget $\Delta = 10$. As can be seen from Table IV, in the test results of the DBLP dataset, the modularity measured by different community detection algorithms for the adversarial graph obtained with a budget of $\Delta = 10$ is not much different from the modularity of the original graph. For the Finance dataset, there is a small difference in the measured results, and the reason is consistent with the analysis in Table III. The results in Table IV show that the modularity of adversarial graphs measured by different community detection algorithms is not significantly different from the modularity measured for the original graph, and it shows that the adversarial graphs obtained by our ACD attack can prevent them from detecting the changes after the attack, so the obtained adversarial graphs can fight against different community detection methods.

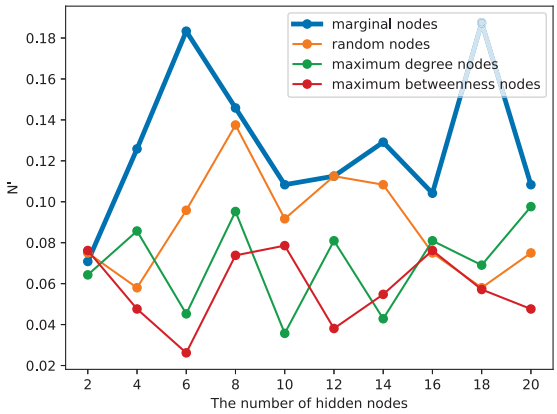


Fig. 6. Evolution of N' as a function of the number of target hidden nodes.

In Table V, Figs. 6, and 7, we can see that for N', at $\Delta = 2$ our ACD attack is not as effective on the marginal node

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on October 27,2026 at 10:37:20 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
PERFORMANCE COMPARISON OF DIFFERENT ATTACKS METHODS UNDER DIFFERENT DATASETS

Data sets	DBLP_small		DBLP_medium		DBLP_large		Finance_small		Finance_medium		Finance_large	
	$N^{'}$	$N^{''}$	$N^{'}$	$N^{''}$	$N^{'}$	$N^{''}$	$N^{'}$	$N^{''}$	$N^{'}$	$N^{''}$	$N^{'}$	$N^{''}$
RTA	0.08237	0.49352	0.96553	0.74219	0.06249	0.44659	0.04351	0.49278	0.02530	0.42670	**0.04739**	0.541272
DICE	0.09804	0.64392	0.10634	0.81439	**0.08361**	0.54769	0.06320	0.64032	0.03249	0.53524	0.03658	0.72110
ACD	**0.10833**	**0.69000**	**0.11770**	**0.84151**	0.08109	**0.57619**	**0.07291**	**0.73499**	**0.03319**	**0.62734**	0.04139	**0.87393**

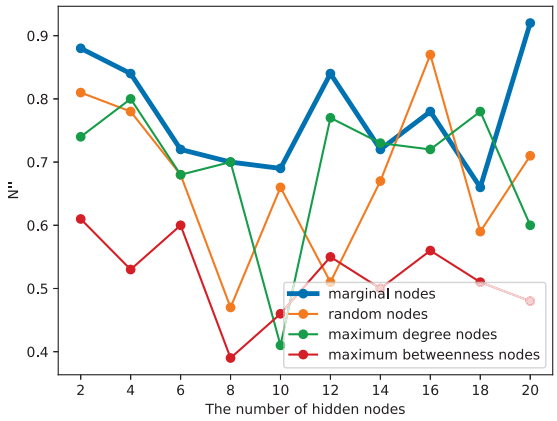


Fig. 7. Evolution of N'' as a function of the number of target hidden nodes.

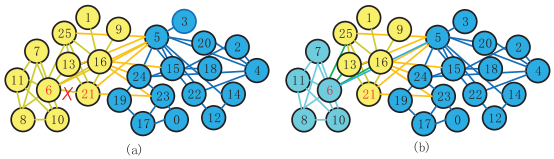


Fig. 8. Case shows how ACD hide marginal individuals. (a) Original graph. (b) Modified graph.

set as other node sets, but when Δ is between 4 and 20, our results are more effective than other node sets, illustrating the effectiveness of our ACD attack to divide marginal nodes into different communities. For N'', budget Δ is between 2 and 12, and our ACD attack on the marginal node sets is more effective. When Δ is between 12 and 20, it is a bit worse than other node selections. On the whole, it can be inferred that the effectiveness of our ACD attack on marginal node sets is better than other node sets, which shows that the hiddenness of marginal community nodes under ACD attack is better than other node selections, and it also illustrates the feasibility of our marginal community node selection algorithm.

The experimental results in Table VI show the performance comparison of different node sets under attack with budget $\Delta = 10$ on all the six datasets. From the experimental results, it can be seen that for the DBLP_small, DBLP_medium, and Finance_large datasets, our ACD attack is more effective on the marginal node set. For instance, compared with maximum selection, the marginal selection improved by 25.0% and 11.5% for N' and N'' on the Finance_large dataset, respectively. Although the value of N' obtained by ACD attacking the randomly selected node set in the Finance_small and Finance_medium datasets is better than the marginal node N' of the set. This is not surprising, because in some cases, the randomly selected node set may be better than the marginal selected node because of the randomness.

Table VII lists the execution time comparison of different attack methods under different datasets. The average execution time of our algorithm is about 8.12% and 4.01% less than baseline RTA and DICE, respectively, on the six datasets.

Table VIII lists the experimental results on the six datasets. All the attacks are based on the same graphs in each dataset, and we experimentally derived values for N' and N''. Among all the methods, ACD achieves the best performance on all the datasets, which demonstrates the superiority of the ACD method. DICE outperforms RTA on all the datasets, probably DICE follows a heuristic and using S^+ to reduce the density of connections would help. For instance, compared with DICE, our ACD improved by 15.36% and 14.8% for N' and N'' on the Finance_small dataset, respectively.

To better understand how ACD works, we target a subgraph of DBLP_small, which consists of 26 nodes under attack, and the subgraph obtained using the marginal node selection algorithm contains two marginal target nodes 6 and 21. We examine how community distribution changes with attack, as shown in Fig. 8. In this figure, different colors represent different community assignments. As we can see, at the beginning the community detection algorithm considers the two targets nodes are in the same community (yellow). When we delete an edge (6, 21), the community detection algorithm divides the subgraph into two parts, so that the two target nodes are divided into different communities, so it cannot handle two targets in one community.

V. CONCLUSION

In social networks, for privacy protection purpose, hiding a set of marginal target individuals into multiple communities has become an important topic which is still open so far. In this work, we propose a deep-graph-learning-based target individuals' hiding method for marginal node pairs, and these node pairs are selected as the intersection ones of variety of community detection methods. To overcome the difficulty that the detection module often does not have explicit feedback in the experiments, we design a new graph-learning-based community detection model based on the widely used GNN and normalized cut. Given the huge search space for selecting suitable candidate edges to modify, we adopt a learning approach and design a suitable graph generator that can satisfy discrete constraints. Then we perturbed a tiny set of links to achieve the marginal user community relationships. Finally, we conducted experiments on six real datasets of DBLP_small, DBLP_medium, DBLP_medium, Finance_small, Finance_medium, and Finance_large and

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on October 27,2026 at 10:37:20 UTC from IEEE Xplore. Restrictions apply.

found that our experimental results are better than nodes with random selection, nodes with the largest degree, and nodes with the largest betweenness methods. The experimental results on the DBLP_large and Finance_large real-world datasets show that our ACD method significantly outperforms other competitors on N' and N'' metrics. Furthermore, we have also conducted corresponding experiments on other classical datasets, such as Football,[1] Karate[1], and Dolphins.[1] The experimental results and conclusions show that they have similar performance to the DBLP and Finance datasets that we use, and we did not the repeat experimental results here. Finally, we do robustness experiments and found that the perturbed graph can defend a variety of ACD methods. Our work has potential applications into real social graphs to locally perturb network structure to achieve the local privacy protection.

As more and more attentions focused on the problem of information over-mined in social networks, many new problems such as privacy leakage and data abuses are emerging. First, although we proposed a new deep-learning-based sensitive marginal user relationship hiding mechanism, the efficiency is a bit limited. Thus, new models and methods of higher efficiency and effectiveness are expected. Second, the relationship among users used in this work is simple (i.e., user links), and the studies on more complicated relationship patterns (e.g., triangle motif) will be more interesting. Third, current work mainly focuses on nonoverlapping community detection algorithms, but there are also many overlapping community scenarios and applications, on which there is a lack of study so far.

REFERENCES

[1] S. Fortunato, "Community detection in graphs," *Phys. Rep.*, vol. 486, nos. 3–5, pp. 75–174, Feb. 2010.

[2] Y. Dourisboure, F. Geraci, and M. Pellegrini, "Extraction and classification of dense communities in the web," in *Proc. 16th Int. Conf. World Wide Web (WWW)*, 2007, pp. 461–470.

[3] G. W. Flake, S. Lawrence, C. L. Giles, and F. M. Coetzee, "Self-organization and identification of web communities," *Computer*, vol. 35, no. 3, pp. 66–70, Mar. 2002.

[4] L. Freeman, "The development of social network analysis," *A Study Sociol. Sci.*, vol. 1, no. 687, pp. 159–167, 2004.

[5] J. Moody and D. R. White, "Structural cohesion and embeddedness: A hierarchical concept of social groups," *Amer. Sociol. Rev.*, vol. 68, no. 1, pp. 103–127, 2003.

[6] L. Akoglu, H. Tong, and D. Koutra, "Graph based anomaly detection and description: A survey," *Data Mining Knowl. Discovery*, vol. 29, no. 3, pp. 626–688, 2015.

[7] X. Chen, Z. Jiang, H. Li, J. Ma, and P. S. Yu, "Community hiding by link perturbation in social networks," *IEEE Trans. Computat. Social Syst.*, vol. 8, no. 3, pp. 704–715, Jun. 2021.

[8] S. Nagaraja, "The impact of unlinkability on adversarial community detection: Effects and countermeasures," in *Proc. Int. Symp. Privacy Enhancing Technol. Symp.*, vol. 6205. Springer, 2010, pp. 253–272.

[9] M. Waniek, T. P. Michalak, M. J. Wooldridge, and T. Rahwan, "Hiding individuals and communities in a social network," *Nature Hum. Behav.*, vol. 2, no. 2, pp. 139–147, Jan. 2018.

[10] J. Li, H. Zhang, Z. Han, Y. Rong, H. Cheng, and J. Huang, "Adversarial attack on community detection by hiding individuals," in *Proc. Web Conf.*, 2020, pp. 917–927.

[11] Z. Yu, Z. Jiang, J. Ma, and Y. Zheng, "Safeness-based community penetration," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 7, pp. 2690–2694, Jul. 2021.

[1] http://www-personal.umich.edu/~mejn/netdata/

[12] G. Qin and L. Gao, "Spectral clustering for detecting protein complexes in protein–protein interaction (PPI) networks," *Math. Comput. Model.*, vol. 52, nos. 11–12, pp. 2066–2074, 2010.

[13] K. Berahmand, A. Bouyer, and M. Vasighi, "Community detection in complex networks by detecting and expanding core nodes through extended local similarity of nodes," *IEEE Trans. Computat. Social Syst.*, vol. 5, no. 4, pp. 1021–1033, Dec. 2018.

[14] B. Saoud and A. Moussaoui, "Node similarity and modularity for finding communities in networks," *Phys. A, Stat. Mech. Appl.*, vol. 492, pp. 1958–1966, Feb. 2018.

[15] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 69, Feb. 2004, Art. no. 026113.

[16] M. Coscia, F. Giannotti, and D. Pedreschi, "A classification for community discovery methods in complex networks," *Stat. Anal. Data Mining, ASA Data Sci. J.*, vol. 4, no. 5, pp. 512–546, 2011.

[17] A. Clauset, "Finding local community structure in networks," *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 72, no. 2, Aug. 2005, Art. no. 026132.

[18] Z. B. Imtiaz, A. Manzoor, S. U. Islam, M. A. Judge, K.-K.-R. Choo, and J. J. P. C. Rodrigues, "Discovering communities from disjoint complex networks using multi-layer ant colony optimization," *Future Gener. Comput. Syst.*, vol. 115, pp. 659–670, Feb. 2021.

[19] I. Inuwa-Dutse, M. Liptrott, and I. Korkontzelos, "A multilevel clustering technique for community detection," *Neurocomputing*, vol. 441, pp. 64–78, Jun. 2021.

[20] G. Palla, I. Derényi, I. Farkas, and T. Vicsek, "Uncovering the overlapping community structure of complex networks in nature and society," *Nature*, vol. 435, no. 7043, pp. 814–818, Jun. 2005.

[21] P. Ronhovde and Z. Nussinov, "Multiresolution community detection for megascale networks by information-based replica correlations," *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 80, no. 1, Jul. 2009, Art. no. 016109.

[22] Z. Zhang, Y. Gong, K. Wang, and J. Gu, "A survey of overlapping community detection based on multi-label propagation," in *Proc. 12th IEEE Conf. Ind. Electron. Appl. (ICIEA)*, Jun. 2017, pp. 995–999.

[23] J. J. Whang, D. F. Gleich, and I. S. Dhillon, "Overlapping community detection using neighborhood-inflated seed expansion," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 5, pp. 1272–1284, May 2016.

[24] K. Asmi, D. Lotfi, and A. Abarda, "The greedy coupled-seeds expansion method for the overlapping community detection in social networks," *Computing*, vol. 104, no. 2, pp. 295–313, Feb. 2022.

[25] K. Liu and E. Terzi, "Towards identity anonymization on graphs," in *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)*, 2008, pp. 93–106.

[26] M. Yuan, L. Chen, P. S. Yu, and T. Yu, "Protecting sensitive labels in social network data anonymization," *IEEE Trans. Knowl. Data Eng.*, vol. 25, no. 3, pp. 633–647, Mar. 2011.

[27] B. Zhou and J. Pei, "Preserving privacy in social networks against neighborhood attacks," in *Proc. IEEE 24th Int. Conf. Data Eng.*, Apr. 2008, pp. 506–515.

[28] X. Zheng, Z. Cai, G. Luo, L. Tian, and X. Bai, "Privacy-preserved community discovery in online social networks," *Future Gener. Comput. Syst.*, vol. 93, pp. 1002–1009, Apr. 2019.

[29] V. Fionda and G. Pirrò, "Community deception or: How to stop fearing community detection algorithms," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 4, pp. 660–673, Apr. 2018.

[30] J. Chen et al., "Ga-based Q-attack on community detection," *IEEE Trans. Computat. Social Syst.*, vol. 6, no. 3, pp. 491–503, May 2019.

[31] V. Konda and J. Tsitsiklis, "Actor-critic algorithms," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 12, 1999, pp. 1–7.

[32] E. W. Dijkstra, "A note on two problems in connection with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[33] M. Girvan and N. Me, "Network of American football games between division ia colleges during regular season fall 2000," *Proc. Nat. Acad. Sci. USA*, vol. 99, pp. 7821–7826, Jan. 2002.

[34] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Stat. Mech., Theory Exp.*, vol. 2008, no. 10, Oct. 2008, Art. no. P10008.

[35] M. Girvan and M. E. J. Newman, "Community structure in social and biological networks," *Proc. Nat. Acad. Sci. USA*, vol. 99, no. 12, pp. 7821–7826, Apr. 2002.

[36] M. Rosvall and C. T. Bergstrom, "Maps of random walks on complex networks reveal community structure," *Proc. Nat. Acad. Sci. USA*, vol. 105, no. 2, pp. 1118–1123, 2008.

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on October 27,2026 at 10:37:20 UTC from IEEE Xplore. Restrictions apply.

[37] J. Shi and J. Malik, "Normalized cuts and image segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 22, no. 8, pp. 888–905, Aug. 2000.
[38] M. Tang, A. Djelouah, F. Perazzi, Y. Boykov, and C. Schroers, "Normalized cut loss for weakly-supervised CNN segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 1818–1827.
[39] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
[40] J. Klicpera, A. Bojchevski, and S. Günnemann, "Predict then propagate: Graph neural networks meet personalized pagerank," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019, pp. 1–15.
[41] R. Sutton and A. Barto, "Reinforcement learning: An introduction (adaptive computation and machine learning)," *IEEE Trans. Neural Netw.*, vol. 9, no. 5, p. 1054, Sep. 1998.
[42] M. Waniek, T. P. Michalak, M. J. Wooldridge, and T. Rahwan, "Hiding individuals and communities in a social network," *Nature Human Behaviour*, vol. 2, no. 2, pp. 139–147, 2018.
[43] D. Zügner, A. Akbarnejad, and S. Günnemann, "Adversarial attacks on neural networks for graph data," in *Proc. 24th Int. Joint Conf. Artif. Intell.*, Aug. 2019, pp. 2847–2856.



Cong Chen received the B.S. degree from the Shaanxi University of Science and Technology, Xi'an, China, in 2019. He is currently pursuing the M.S. degree with the School of Cyber Engineering, Xidian University, Xi'an.

His current research interests include social network analysis and privacy protection.



Zhongyuan Jiang received the B.S. and Ph.D. degrees from Beijing Jiaotong University, Beijing, China, in 2009 and 2013, respectively.

He is currently an Associate Professor with the School of Cyber Engineering, Xidian University, Xi'an, China. His research interests include privacy preserving, social computing, urban computing, and network functions' virtualization.



Jianfeng Ma (Member, IEEE) received the B.S. degree from Shaanxi Normal University, Xi'an, China, in 1985, and the Ph.D. degree from Xidian University, Xi'an, in 1995.

He is a Professor with the School of Cyber Engineering, Xidian University and the Director of the State Key Laboratory of Integrated Services Networks, Xidian University. His research interests include cryptography, wireless network security, data security, and mobile intelligent system security.

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on October 27,2026 at 10:37:20 UTC from IEEE Xplore. Restrictions apply.